

VDM-SL

VDM-SL

- VDM is a “model-oriented approach”
- An explicit model of the state of an abstract machine is given, and operations are defined in terms of this state.
- Operations are defined in a precondition and postcondition style.
- Each operation has an associated proof obligation to ensure that if the precondition is true, then the operation preserves the system invariant.
- The initial state itself is, of course, required to satisfy the system invariant.

Comparative Analysis between VDM and Z

Characteristic	VDM	Z
Mathematical Basis	• Set Theory • First Order Predicate Calculus	• Set Theory • First Order Predicate Calculus
Model or Property Oriented	Model-Oriented	Model-Oriented
Specified Properties	Behavior (functionality) and design of sequential programs	Behavior (functionality) and design of sequential programs
Design Aspects Not Supported	• System Timing • Concurrency • User Interface	• System Timing • Concurrency • User Interface
Page Appearance Differences	Heavily Keyword oriented (e.g. pre-, post-, invariants)	Boxes or Schemas
Structuring Mechanisms	None	Schema Calculus which allows various schemas to be combined to form new schemas
Specification of State Changes	• Before: hooked variables • After: unhooked variables	• Before: undecorated variables • After: primed variables
Identification of Inputs and Outputs	No explicit way of specifying	• Inputs: variable names ending in “?” • Outputs: variable names ending in “!”
Distinguishing Constants and Variables	Keyword wr for variables that can change. Keyword rd for variables that are read only	D for variables that can change. X for variables that are read only.

VDM-SL

A VDM specification consists of

- Type definitions
- State Definitions
- Invariant for the system
- Definition of the operations of the system

Built-in Types in VDM

Set	Name	Elements
B	Boolean	{ true, false }
N	Naturals	{ 0, 1, ... }
N ₁	Naturals (excluding 0)	{ 1, 2, ... }
Z	Integers	{ ...-1, 0, 1, ... }
Q	Rational numbers	{ $p/q : p, q \in \mathbb{Z} \text{ } q \neq 0$ }
R	Real numbers	

Example

- Consider a simple library system that allows borrowing and returning of books. The data types for the library system are first defined, and the operation to borrow a book is then defined.
- It is assumed that the state is made up of three sets, and these are the set of books on the shelf, the set of books which are borrowed, and the set of missing books.
- These sets are mutually disjoint.
- The effect of the operation to borrow a book is to remove the book from the set of books on the shelf and to add it to the set of borrowed books.

Example-Borrow Book-Library Management System

types

$Bks = Bkd-id \text{ set}$

state *Library* of

$On-shelf : Bks$

$Missing : Bks$

$Borrowed : Bks$

inv $mk-Library (os, mb, bb) \triangleq is-disj(\{os, mb, bb\})$

end

$borrow (b:Bkd-id)$

ex wr $on-shelf, borrowed : Bks$

pre $b \in on-shelf$

post $on-shelf = on-shelf^- - \{b\} \wedge$

$borrowed = borrowed^- \cup \{b\}$

Example-Explanation

- The notation Bkd-id set specifies that Bks is a set of Bkd-ids, e.g. $Bks = \{b1, b2, \dots, bn\}$.
- The invariant specifies the property that must remain true for the library system.
- Borrow operation is defined using preconditions and postconditions.
- The notation “ext wr” indicates that the borrow operation affects the state
- The notation “ext rd” indicates an operation that does not affect the state.

Sequences

- Sequences are used frequently in VDM specifications.
- A sequence is a collection of items that are ordered in a particular way.
- Duplicate items are allowed for sequences, whereas duplicate elements are not meaningful for sets.

Sequences

- A sequence of elements $x_1, x_2, \dots, x_n$ is denoted by $[x_1, x_2, \dots, x_n]$, and the empty sequence is denoted by $[]$.
- Given a set S , then S^* denotes the set of all finite sequences constructed from the elements of S .
- The length of a sequence is given by the `len` operator:

`len [] = 0`

`len [1, 2, 6] = 3`

Sequences

- The `hd` operation gives the first element of the sequence. It is applied to non-empty sequences only:

$$\text{hd } [x] = x$$

$$\text{hd } [x, y, z] = x$$

Sequences

- The `tl` operation gives the remainder of a sequence after the first element of the sequence has been removed. It is applied to non-empty sequences only:

```
tl [x] = []  
tl [x, y, z] = [y, z]
```

Sequences

- The `elems` operation gives the elements of a sequence. It is applied to both empty and non-empty sequences:

```
elems [] = { }
```

```
elems [x, y, z] = {x, y, z}
```

Sequences

- The `idx` operation is applied to both empty and non-empty sequences. It returns the set $\{1, 2, \dots, n\}$ where n is the number of elements in the sequence.

```
idx [] = {}  
idx [x, y, z] = {1, 2, 3}  
idx s = {1, .. len s}
```

Sequence

- Two sequences may be joined together by the concatenation operator:

$$[] \curvearrowright [] = []$$

$$[x, y, z] \curvearrowright [a, b] = [x, y, z, a, b]$$

$$[x, y] \curvearrowright [] = [x, y]$$

Specification For Stack of Integer

A stack of (up to 100) integers is specified as:

state Z-stack of	
stk : $\mathbb{Z}^*$	
inv-Z-stack (stk) Δ $ \text{len } \text{stk} \leq 100$	
init-mk-Z-stack (stk) Δ	
stk = []	
end	

Push Operation

- The push operation is then specified in terms of preconditions/postconditions as :

push ($z : \mathbb{Z}$)
pre len stk < 100
post stk = [z] $\cap$ stk[←]